

BI

Documentacao Tecnica do Dashboard

BI ERP + MES Engenharia de Producao N2

ARQUIVO	BI_Engenharia_Producao.html
ELABORADO	09 de junho de 2026
DISCIPLINA	Engenharia de Producao - N2

Sumário

- 1. Visão Geral do Projeto**
- 2. Contexto Tecnológico: ERP e MES**
- 3. Fase 1 — Conexão e Extração de Dados**
 - 3.1 Conexão com o ERP (PostgreSQL)
 - 3.2 Queries SQL e Agregações
 - 3.3 Conexão com o MES (MongoDB)
 - 3.4 Pipelines de Agregação MongoDB
- 4. Fase 2 — Arquitetura do Diagrama de Relacionamento**
 - 4.1 Estrutura do ERP: 26 tabelas em 4 módulos
 - 4.2 Chaves Primárias e Estrangeiras
 - 4.3 Estrutura do MES: 6 coleções MongoDB
- 5. Fase 3 — Estrutura do Documento HTML**
 - 5.1 Tecnologias e bibliotecas utilizadas
 - 5.2 Padrão SECTION_INIT / SECTION_DONE
 - 5.3 Organização da navegação e seções
- 6. Fase 4 — Módulos do Dashboard**
- 7. Dados Embarcados vs. Conexão em Tempo Real**
- 8. Considerações Finais**

1. Visão Geral do Projeto

O arquivo **BI_Engenharia_Producao.html** é um dashboard de Business Intelligence completamente auto-contido: um único arquivo HTML que concentra toda a lógica de visualização, os dados e a navegação, sem depender de um servidor externo em tempo de execução. Foi desenvolvido para a disciplina de **Engenharia de Produção (N2)** com o objetivo de integrar informações de dois sistemas distintos — um ERP baseado em PostgreSQL e um MES baseado em MongoDB — em uma interface unificada de análise.

O processo de elaboração foi dividido em quatro fases principais, descritas ao longo deste documento:

Fase	Descrição
Fase 1	Conexão aos bancos de dados e extração das consultas
Fase 2	Modelagem e documentação da arquitetura relacional
Fase 3	Construção da estrutura HTML, CSS e JavaScript
Fase 4	Implementação dos módulos de visualização (gráficos e tabelas)

Resultado final: um dashboard com 13 seções navegáveis cobrindo KPIs executivos, produção, máquinas, qualidade, clientes, fornecedores, estoque, receita, DRE, relatórios tabelados, diagramas de relacionamento e metodologia — tudo renderizado diretamente no navegador, sem backend ativo.

2. Contexto Tecnológico: ERP e MES

A empresa fictícia modelada no projeto fabrica produtos hidráulicos (bombas e válvulas) e opera com dois sistemas de informação complementares, cada um escolhido de acordo com a natureza dos dados que precisa gerenciar:

Sistema	Banco de Dados	Função	Escopo
ERP	PostgreSQL	Planejamento de recursos empresariais	Cadastros, comercial, produção e financeiro
MES	MongoDB	Execução e monitoramento da produção	OEE, paradas, manutenções e qualidade

A escolha de bancos de dados diferentes foi intencional: o ERP lida com dados estruturados e altamente relacionais — pedidos, faturas, ordens de produção — o que torna o **PostgreSQL** ideal. O MES, por sua vez, captura eventos em tempo real das máquinas, com estrutura variável (dados de sensores), onde o **MongoDB** com documentos JSON oferece maior flexibilidade.

Produtos monitorados — 8 SKUs

SKU	Descrição
BMB-100	Bomba Centrífuga 1cv
BMB-200	Bomba Centrífuga 2cv
BMB-300	Bomba Submersa 3cv
CJM-500	Conjunto Pressurizador
CJM-700	Conjunto Booster Duplo
VLV-050	Válvula de Esfera ½"
VLV-075	Válvula de Esfera ¾"
VLV-100	Válvula de Gaveta 1"

Volume de dados — exercício 2025

Indicador	Valor
Ordens de Produção (OPs)	96
Pedidos comerciais	152

Indicador	Valor
Clientes	12
Fornecedores	6
Receita Total Bruta	R\$ 562.314.028,70
Peças processadas (MES)	606.462
Taxa de refugo global	1,64%
OEE médio das máquinas	78,9%
Ordens de manutenção	207

3. Fase 1 — Conexão e Extração de Dados

Antes de construir o HTML, foi necessário conectar-se aos bancos de dados, executar as consultas e transformar os resultados no formato JSON que seria embarcado no arquivo. Esta etapa é análoga ao que uma aplicação BI real realiza em tempo de execução — aqui, porém, os dados foram coletados uma única vez e 'congelados' dentro do HTML.

3.1 Conexão com o ERP (PostgreSQL)

A conexão com o PostgreSQL foi estabelecida em Python utilizando a biblioteca `psycopg2`, o driver padrão de mercado para integração Python–PostgreSQL. O processo segue o padrão de conexão relacional clássico: cria-se um objeto de conexão e, a partir dele, um *cursor* — o canal que permite enviar comandos SQL e receber os resultados de volta como listas de tuplas ou dicionários Python.

PYTHON · PSYCOPG2 — CONEXÃO COM POSTGRESQL

```
import psycopg2
import json, pandas as pd

# Parâmetros de conexão com o banco ERP
conn = psycopg2.connect(
    host='localhost',
    port=5432,
    dbname='erp_producao',
    user='erp_user',
    password='senha_segura'
)

# O cursor é o canal de comunicação com o banco.
# Por meio dele, os comandos SQL são enviados e
# os resultados retornam como listas Python.
cursor = conn.cursor()
```

NOTA `psycopg2` implementa o protocolo binário do PostgreSQL (`libpq`), tornando a comunicação eficiente. Ele converte automaticamente os tipos do banco (`int`, `float`, `datetime`, etc.) para seus equivalentes Python.

3.2 Queries SQL e Agregações

Cada conjunto de dados exibido no dashboard é resultado de uma query SQL executada no PostgreSQL. As consultas utilizam extensivamente as funções de agregação do SQL (`GROUP BY`, `SUM`, `COUNT`, `AVG`) para consolidar os dados brutos das tabelas transacionais em resumos prontos para visualização.

Query 1 — Receita líquida mensal

Cruza as tabelas `pedidos`, `itens_pedido` e `produtos`. Calcula a receita líquida mês a mês para o ano de 2025, já descontando o percentual de desconto de cada item.

```
SQL · ERP — RECEITA MENSAL AGREGADA

-- Receita líquida por mês (12 meses de 2025)
SELECT
    TO_CHAR(p.data_pedido, 'YYYY-MM') AS ano_mes,
    SUM(ip.qtd_vendida
        * ip.preco_unitario
        * (1 - ip.desconto_pct / 100.0)) AS receita_liquida
FROM pedidos p
JOIN itens_pedido ip ON ip.pedido_id = p.pedido_id
WHERE p.status = 'CONFIRMADO'
    AND EXTRACT(YEAR FROM p.data_pedido) = 2025
GROUP BY TO_CHAR(p.data_pedido, 'YYYY-MM')
ORDER BY ano_mes;
```

Query 2 — Saldo de estoque atual

Lê a tabela `movimentacoes_estoque` e calcula o saldo de cada produto somando entradas e subtraindo saídas.

```
SQL · ERP — ESTOQUE ATUAL POR PRODUTO

-- Saldo de estoque = entradas - saídas
SELECT
    pr.sku,
    pr.nome_produto,
    SUM(CASE
        WHEN me.tipo_mov = 'ENTRADA'
        THEN me.quantidade
        ELSE -me.quantidade
    END) AS estoque_atual
FROM movimentacoes_estoque me
JOIN produtos pr ON pr.produto_id = me.produto_id
GROUP BY pr.sku, pr.nome_produto;
```

Query 3 — Ordens de produção por status

Conta quantas ordens de produção existem em cada status na tabela `ordens_producao`.

```
SQL · ERP — ORDENS DE PRODUÇÃO POR STATUS
```

```
SELECT status, COUNT(*) AS total  
FROM ordens_producao  
GROUP BY status  
ORDER BY total DESC;
```

Os resultados de cada query são convertidos para lista de dicionários via pandas (`df.to_dict("records")`) e depois serializados com `json.dumps(...)`. O JSON resultante é inserido diretamente no JavaScript do HTML como uma constante.

3.3 Conexão com o MES (MongoDB)

O sistema MES armazena os dados de chão de fábrica em coleções MongoDB. A biblioteca utilizada é o **pymongo**, o driver oficial Python para MongoDB. A diferença fundamental em relação ao PostgreSQL é que, no MongoDB, não existem tabelas nem schemas fixos: os dados são **documentos JSON** (internamente BSON) organizados em coleções.

```
PYTHON · PYMONGO — CONEXÃO COM MONGODB

from pymongo import MongoClient

# Conexão com o servidor MongoDB local
client = MongoClient('mongodb://localhost:27017/')
db = client['mes_fabrica']

# Cada variável aponta para uma coleção diferente
apontamentos = db['apontamentos_producao']
paradas       = db['registro_paradas']
manutencoes   = db['ordens_manutencao']
inspecoes     = db['inspecoes_qualidade']
```

Exemplo de documento — coleção apontamentos_producao

O documento abaixo ilustra a estrutura típica de um registro de produção capturado pelo MES. Note o campo **op_id**, que funciona como elo de integração com o ERP (referencia a ordem de produção do PostgreSQL).

```
DOCUMENTO MONGODB · APONTAMENTOS_PRODUCAO

{
  '_id':           ObjectId('64a1...'),
  'recurso':       'MON-01',           // identificador da máquina
  'op_id':         'OP-00001',        // liga ao ERP (chave de integração)
  'turno':         1,
  'qtd_produzida': 480,
  'qtd_refugo':    8,
  'tempo_operando': 430,               // em minutos
  'tempo_planejado': 480,
  'timestamp':     ISODate('2025-01-15T08:00:00Z')
}
```

3.4 Pipelines de Agregação MongoDB

No MongoDB, o equivalente ao **GROUP BY** do SQL é o **Aggregation Pipeline**: uma sequência de estágios que transformam os documentos progressivamente. Cada estágio recebe os documentos do

anterior, processa e passa adiante.

Estágio	Equivalente SQL	Função
\$match	WHERE	Filtra documentos por condição
\$group	GROUP BY	Agrupar e calcular acumuladores (sum, avg, count)
\$sort	ORDER BY	Ordena os resultados
\$project	SELECT	Seleciona e renomeia campos
\$lookup	JOIN	Une documentos de outra coleção

Pipeline 1 — Cálculo do OEE por máquina

O OEE (Overall Equipment Effectiveness) é calculado como o produto de três fatores: **Disponibilidade** × **Performance** × **Qualidade**. O pipeline abaixo agrega os apontamentos de cada máquina e calcula os dois componentes disponíveis diretamente no banco.

PYTHON · PYMONGO — PIPELINE OEE POR MÁQUINA

```
pipeline_oe = [
    { '$group': {
        '_id': '$recurso',
        'total_planejado': { '$sum': '$tempo_planejado' },
        'total_operando': { '$sum': '$tempo_operando' },
        'total_boas': { '$sum': '$qtd_produzida' },
        'total_refugo': { '$sum': '$qtd_refugo' },
        # Disponibilidade: tempo_operando / tempo_planejado
        'disponibilidade': { '$avg': {
            '$divide': [ '$tempo_operando', '$tempo_planejado' ] } },
        # Qualidade: (produzidas - refugo) / produzidas
        'qualidade': { '$avg': {
            '$divide': [
                { '$subtract': [ '$qtd_produzida', '$qtd_refugo' ] },
                '$qtd_produzida'
            ] } }
    } },
    { '$sort': { '_id': 1 } }
]

resultado_oe = list(apontamentos.aggregate(pipeline_oe))
```

Pipeline 2 — Paradas por motivo

Agrupa os registros da coleção `registro_paradas` pelo campo motivo, somando a duração total e contando o número de ocorrências de cada tipo.

```
PYTHON · PYMONGO — PARADAS POR MOTIVO

pipeline_paradas = [
    { '$group': {
        '_id': '$motivo',
        'total_min': { '$sum': '$duracao_min' },
        'ocorrencias': { '$sum': 1 }
    } },
    { '$sort': { 'total_min': -1 } } # mais impactantes primeiro
]
```

Após a execução de todos os pipelines, os resultados são serializados com `json.dumps(..., ensure_ascii=False, default=str)` e inseridos dentro de constantes JavaScript no HTML.

4. Fase 2 — Arquitetura do Diagrama de Relacionamento

O diagrama de relacionamento é a representação visual das tabelas (ERP) e coleções (MES), mostrando como elas se conectam entre si. No HTML, esses diagramas são renderizados em **SVG puro via JavaScript** — um renderer customizado que desenha as caixas das tabelas, os campos e as linhas de relacionamento dinamicamente, sem depender de imagens estáticas.

4.1 Estrutura do ERP: 26 tabelas em 4 módulos

O ERP foi modelado em quatro módulos funcionais, cada um com uma cor distinta no diagrama. A organização reflete a hierarquia de processos empresariais:

Módulo	Tabelas	Descrição
Cadastros (azul)	clientes, enderecos_cliente, contatos_cliente, fornecedores, enderecos_fornecedor, produtos, componentes, lista_materiais	Dados mestres da empresa. Raramente mudam; referenciados por todos os outros módulos.
Comercial (verde)	pedidos, itens_pedido, nota_fiscal, itens_nf	Ciclo de venda: do pedido do cliente à emissão da nota fiscal.
Produção (amarelo)	ordens_producao, roteiros_producao, operacoes_roteiro, recursos_producao, compras, itens_compra	Planejamento e execução da fabricação. Liga o pedido à OP e à compra de insumos.
Financeiro/Estoque (roxo)	movimentacoes_estoque, saldo_estoque, contas_receber, contas_pagar, lancamentos_contabeis, plano_contas, centros_custo, parametros_financeiros	Controle financeiro e de estoque. Registra entradas/saídas e lançamentos contábeis.

4.2 Chaves Primárias e Estrangeiras

O diagrama diferencia visualmente dois tipos de campos, usando cores distintas para facilitar a leitura dos relacionamentos:

Notação	Tipo	Cor no diagrama	Significado
PK	Chave Primária	#dc2626 (vermelho)	Identificador único da tabela. Ex.: cliente_id em clientes
FK	Chave Estrangeira	#2563eb (azul)	Referência a outra tabela. Ex.: cliente_id em pedidos aponta para clientes

Principais relacionamentos entre tabelas

Tabela Pai	Chave	Tabela Filha	Cardinalidade
clientes	cliente_id	pedidos	1:N — um cliente, muitos pedidos
pedidos	pedido_id	itens_pedido	1:N — um pedido, muitos itens
ordens_producao	op_id	roteiros_producao	1:1 — cada OP tem um roteiro
produtos	produto_id	lista_materiais	1:N — um produto, muitos componentes
produtos	produto_id	movimentacoes_estoque	1:N — um produto, N movimentações
fornecedores	fornecedor_id	compras	1:N — um fornecedor, N compras

4.3 Estrutura do MES: 6 coleções MongoDB

O MES utiliza documentos JSON em vez de tabelas. O vínculo com o ERP é feito pelo campo **op_id**, que corresponde ao identificador da ordem de produção no PostgreSQL — este é o **ponto de integração entre os dois sistemas**.

Coleção	Campos principais	Propósito
apontamentos_producao	recurso, op_id, turno, qtd_produzida, qtd_refugo, tempo_operando, tempo_planejado	Registros de produção por turno e máquina
registro_paradas	recurso, motivo, duracao_min, turno, data	Paradas planejadas e não planejadas por máquina
ordens_manutencao	maquina, tipo (CORRETIVA/PREVENTIVA), duracao_min, custo	Histórico de manutenções e seus custos
inspecoes_qualidade	produto_sku, tipo defeito, qtd_amstras, qtd_defeitos, aprovado	Resultados de inspeção do controle de qualidade
parametros_maquina	recurso, parametro, valor, unidade, timestamp	Leituras de sensores IoT (temperatura, pressão, vibração)
calibracoes	instrumento, data_calibracao, resultado, validade	Rastreabilidade de instrumentos de medição

5. Fase 3 — Estrutura do Documento HTML

5.1 Tecnologias e bibliotecas utilizadas

Todas as dependências externas são carregadas via CDN (cdn.jsdelivr.net). Isso significa que o arquivo HTML funciona de forma portátil para os dados, mas os estilos e gráficos exigem conexão com a internet para carregar as bibliotecas.

Tecnologia	Versão	Função no projeto
Bootstrap	5.3.2	Framework CSS para layout responsivo (grid, cards, badges, tabelas)
Bootstrap Icons	1.11.3	Ícones SVG para a barra lateral e títulos de seção
Chart.js	4.4.x	Renderização de gráficos: barras, linhas, pizza, rosca
SVG nativo	—	Diagramas de relacionamento desenhados via JavaScript puro
JavaScript ES6+	—	Lógica de navegação, renderização lazy e processamento do JSON
CSS Custom Props.	—	Variáveis CSS para cores e dimensões do layout (--sw, --ac, etc.)

5.2 Padrão SECTION_INIT / SECTION_DONE — Lazy Loading

Uma decisão de arquitetura importante foi implementar **inicialização preguiçosa** (lazy loading) dos gráficos. Como o arquivo contém mais de 30 gráficos Chart.js, inicializar todos na carga da página seria lento e desperdiçaria recursos. O padrão adotado garante que os gráficos de uma seção são criados apenas quando o usuário navega até ela pela primeira vez.

```
JAVASCRIPT · PADRÃO SECTION_INIT / SECTION_DONE

// 1. Dicionário de funções de inicialização (uma por seção)
const SECTION_INIT = {};
// Set que registra quais seções já foram inicializadas
const SECTION_DONE = new Set();

// 2. Função de navegação – só inicializa na primeira visita
function show(sectionName) {
    // Oculta seção atual e exibe a nova
    document.querySelectorAll('.page-section')
        .forEach(el => el.classList.remove('active'));
    document.getElementById(`sec-${sectionName}`)
        .classList.add('active');

    // Cria os gráficos apenas se ainda não foram criados
    if (!SECTION_DONE.has(sectionName) && SECTION_INIT[sectionName]) {
        SECTION_INIT[sectionName](); // executa a função de init
        SECTION_DONE.add(sectionName); // marca como concluída
    }
}

// 3. Cada seção declara sua própria função de inicialização
SECTION_INIT['dashboard'] = () => {
    mkChart('c_recMensal', 'bar', ...); // cria gráficos ao visitar
};
```

NOTA Uma vez criados, os gráficos ficam 'marcados' no Set `SECTION_DONE` e não são recriados nas visitas seguintes à mesma seção.

5.3 Organização da navegação e seções

ID da seção	Título	Fonte dos dados	Tipo de visualização
dashboard	Dashboard Executivo	ERP + MES	KPIs + 4 gráficos
producao	Produção (ERP)	ERP	KPIs + 3 gráficos + tabela
maquinas	Máquinas / MES	MES	4 gráficos (OEE, paradas, manutenção)
qualidade	Qualidade	MES	4 gráficos (defeitos, aprovações)
clientes	Clientes	ERP	2 gráficos + tabela
fornecedores	Fornecedores	ERP	2 gráficos + tabela

ID da seção	Título	Fonte dos dados	Tipo de visualização
estoque	Estoque	ERP	2 gráficos + tabela
financeiro	Receita & Vendas	ERP	4 gráficos + tabela
dre	DRE	ERP	Tabela DRE + 2 gráficos
relatorios	Relatórios	ERP + MES	5 tabelas detalhadas
tabelas	Tabelas de Dados	ERP + MES	5 abas com dados brutos
diagramas	Diagramas	—	SVG renderizado via JS
metodologia	Metodologia	—	Texto descritivo

6. Fase 4 — Módulos do Dashboard

Cada módulo do dashboard é alimentado por um subconjunto do objeto JSON embarcado no JavaScript. O fluxo de dados segue sempre o mesmo padrão:

JSON embarcado → Cálculo JavaScript → Chart.js ou tabela HTML

Exemplo — Gráfico de receita mensal

Os dados chegam na constante `ERP.receita_mensal`, que é um array de objetos `{ano_mes, receita_liquida}` gerado pela query SQL da seção 3.2. A função auxiliar `mkChart()` é um wrapper criado no projeto para simplificar a criação de gráficos Chart.js, evitando repetir configurações como responsividade, fontes e formatação de eixos.

```
JAVASCRIPT · GRÁFICO DE RECEITA MENSAL (CHART.JS)
// ERP.receita_mensal veio do PostgreSQL via query SQL
const rm = ERP.receita_mensal; // array de {ano_mes, receita_liquida}

mkChart(
  'c_recMensal', // id do <canvas> no HTML
  'bar',         // tipo de gráfico
  rm.map(r => r.ano_mes), // labels: eixo X (meses)
  [{
    label: 'Receita Líquida',
    data: rm.map(r => +r.receita_liquida), // eixo Y (R$)
    backgroundColor: 'rgba(37,99,235,.65)'
  }],
  { scales: { y: { ticks: { callback: v => brl(v) } } } }
);
// brl() é uma função auxiliar que formata o valor em R$
```

Cálculo do OEE no frontend

Os dados do MES (`MES.oeo_por_maquina`) chegam como resultado do pipeline MongoDB da seção 3.4. Os três componentes do OEE são calculados e plotados separadamente em um gráfico de barras agrupadas, permitindo comparar máquina a máquina.

JAVASCRIPT · GRÁFICO DE OEE POR MÁQUINA

```
const oee = MES.oee_por_maquina; // resultado do pipeline MongoDB

mkChart('c_ooo', 'bar',
  oee.map(o => o._id), // labels: nome da máquina
  [
    {
      label: 'Disponib.',
      // disponibilidade vem do $avg calculado no pipeline
      data: oee.map(o => (+o.disponibilidade * 100).toFixed(1))
    },
    {
      label: 'Performance',
      // performance = tempo_operando / tempo_planejado
      data: oee.map(o =>
        (+o.total_operando / Math.max(+o.total_planejado, 1)
        * 100).toFixed(1))
    },
    {
      label: 'Qualidade',
      // qualidade vem do $avg calculado no pipeline
      data: oee.map(o => (+o.qualidade * 100).toFixed(1))
    }
  ],
  { scales: { y: { min: 0, max: 100 } } }
);
```

7. Dados Embarcados vs. Conexão em Tempo Real

Uma decisão arquitetural central do projeto foi **embarcar os dados diretamente no HTML** em vez de conectar-se aos bancos em tempo real. Essa escolha tem vantagens e limitações importantes descritas a seguir.

Aspecto	Dados Embarcados (abordagem adotada)	Conexão em Tempo Real
Portabilidade	Um único arquivo HTML — funciona em qualquer lugar	Requer servidor, banco acessível e rede
Atualização	Requer reexecução do script de extração + rebuild	Dados sempre frescos, automáticos
Segurança	Dados congelados no arquivo — sem credenciais expostas	Requer gestão cuidadosa de credenciais e CORS
Desempenho	Instantâneo — sem latência de rede	Depende da velocidade do banco e da rede
Uso acadêmico	Ideal — demonstra a modelagem sem infraestrutura	Exige ambiente configurado para apresentação

Estrutura dos dados no HTML

Os dados do ERP ficam na constante JavaScript `const ERP = {...}` e os do MES em `const MES = {...}`. Cada chave do objeto corresponde a uma query ou pipeline executado anteriormente:

JAVASCRIPT · ESTRUTURA DAS CONSTANTES ERP E MES NO HTML

// Constantes embarcadas no HTML

```
const ERP = {  
  receita_mensal: [...], // query SQL: receita agrupada por mês  
  producao_erp: [...], // query SQL: OPs por produto e mês  
  // ... demais queries  
};  
  
const MES = {  
  oee_por_maquina: [...], // pipeline MongoDB: OEE por recurso  
  paradas_por_motivo: [...], // pipeline MongoDB: paradas agrupadas  
  // ... demais pipelines  
};
```

NOTA Para um sistema de produção real, a arquitetura seria expandida para incluir um backend (ex.: FastAPI ou Node.js) que consultaria os bancos periodicamente e exporia uma API REST. O HTML funcionaria então como cliente SPA (Single Page Application), usando fetch/axios para carregar os dados dinamicamente.

8. Considerações Finais

O dashboard **BI_Engenharia_Producao.html** demonstra, de forma prática e integrada, como sistemas ERP e MES se complementam na gestão industrial. A escolha de um único arquivo HTML auto-contido foi estratégica para garantir facilidade de compartilhamento e portabilidade total.

Síntese do processo construtivo

Etapa	Descrição
1 · Modelagem do banco	Definição das 26 tabelas PostgreSQL (ERP) e 6 coleções MongoDB (MES), com chaves primárias, estrangeiras e relacionamentos entre módulos.
2 · Carga de dados simulados	Geração de dados realistas para 2025: 96 OPs, 152 pedidos, 12 clientes, 6 fornecedores, 8 produtos, eventos de máquinas e inspeções de qualidade.
3 · Extração e transformação	Execução de queries SQL (psycopg2) e pipelines MongoDB (pymongo), conversão para JSON e inserção nas constantes ERP e MES do HTML.
4 · Construção da interface	Layout Bootstrap 5 com sidebar fixa, 13 seções de conteúdo, CSS customizado e padrão SECTION_INIT para inicialização lazy dos gráficos Chart.js.
5 · Renderização dos diagramas	Substituição de imagens estáticas por um renderer SVG em JavaScript puro, desenhando tabelas, campos PK/FK e linhas de relacionamento dinamicamente.
6 · Validação e correção	Revisão dos valores de estoque (correção de negativos), alinhamento dos dados entre gráficos e tabelas, e ajustes visuais finais.

Este documento foi gerado a partir da análise do código-fonte do arquivo BI_Engenharia_Producao.html e reflete a arquitetura e as decisões técnicas implementadas.

Data de geração: 09/06/2026 · Python 3.12 · ReportLab 4.4 · Formato A4